

LRU Cache — LeetCode 146

Problem

Design a data structure with $O(1)$ average-time operations:

- `LRUCache(int capacity)` — initialize with positive capacity.
- `int get(int key)` — return the value if the key exists, otherwise `-1`.
- `void put(int key, int value)` — insert or update; if the insert exceeds capacity, evict the least recently used entry.

Both `get` and `put` count as "using" a key. Constraints: `1 <= capacity <= 3000`, keys/values are ints, up to $\sim 2 \times 10^5$ calls.

Concepts used

1. **HashMap for $O(1)$ lookup — but the value is the *node*, not the value.** This is the key insight. Mapping `key` $\rightarrow$ `value` leaves you with no way to find the key's place in the recency ordering without searching. Mapping `key` $\rightarrow$ `node` gives you a direct handle on its position.
2. **Doubly linked list as the recency ordering.** Unlink a node from the middle and re-splice it at the front — pure pointer surgery, $O(1)$, no searching, no shifting.
3. **Why doubly, not singly.** Unlinking needs the node's predecessor. With a singly linked list you'd have to walk from the head to find it: $O(n)$. Doubly linked gives you `node.prev` directly.
4. **Sentinel head/tail nodes.** Dummy nodes at both ends eliminate every null check at the boundaries — `insertAtHead` and `unlink` never special-case empty list, first node, or last node.
5. **The recency invariant.** `head.next` is always the most recently used, `tail.prev` the least. Every `get` and `put` ends by restoring this invariant (move-to-head). Eviction is then trivially `tail.prev`.
6. **Evict from both structures.** The classic bug: removing the node from the list but forgetting `map.remove(key)` — the map then resurrects a dead node on the next `get`.
7. **Why the alternatives fail:**
8. FIFO queue + map: `get` must reorder, a queue can't.
9. Array/queue with shift-to-front: the shift is $O(n)$ per operation.
10. Storing an integer "position" in the node: the index goes stale after every insert or move — everything behind the insertion point shifts. Keeping indices correct costs $O(n)$ per op.
11. Heap keyed by timestamp: updates are $O(\log n)$, and finding the key inside the heap needs a separate `key` $\rightarrow$ `index` map anyway.

Approach

Maintain the invariant above. `get` : look up the node, move it to the head, return its value (`-1` on miss). `put` : if the key exists, update the value and move to head (no eviction — size doesn't change); otherwise evict `tail.prev` if at capacity, then insert the fresh node at the head and into the map.

Code

```
import java.util.HashMap;
import java.util.Map;

public class LRUCache {

    private static class Node {
        int key, value;
        Node prev, next;

        Node(int key, int value) {
            this.key = key;
            this.value = value;
        }
    }

    private final int capacity;
    private final Map<Integer, Node> map;
    private final Node head; // sentinel: head.next is the most-recently-used node
    private final Node tail; // sentinel: tail.prev is the least-recently-used node

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.map = new HashMap<>();
        this.head = new Node(-1, -1);
        this.tail = new Node(-1, -1);
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        Node node = map.get(key);
        if (node == null) {
```

```

        return -1;
    }
    moveToHead(node);
    return node.value;
}

public void put(int key, int value) {
    Node node = map.get(key);
    if (node != null) {
        node.value = value;    // update, don't duplicate
        moveToHead(node);
        return;
    }
    if (map.size() == capacity) {
        Node lru = tail.prev;
        unlink(lru);
        map.remove(lru.key);    // evict from BOTH structures
    }
    Node fresh = new Node(key, value);
    map.put(key, fresh);
    insertAtHead(fresh);
}

private void moveToHead(Node node) {
    unlink(node);
    insertAtHead(node);
}

/** O(1): the node IS its own position – no search, no index. */
private void unlink(Node node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

private void insertAtHead(Node node) {
    node.next = head.next;
    node.prev = head;
    head.next.prev = node;
    head.next = node;
}
}

```

Complexity

- Time: $O(1)$ average for `get` and `put` (HashMap ops + a constant number of pointer writes).
- Space: $O(\text{capacity})$ — one node + one map entry per key.

Edge cases that matter

- `get` on a missing key → `-1` (don't forget the return path).
- `put` on an existing key → update value + refresh recency; must **not** evict, size is unchanged.
- Capacity 1: every new `put` evicts the previous entry.
- Eviction must remove from the map too, or `get` resurrects dead nodes.
- Never mutate a node that's still linked without unlinking first (double-insert corrupts the list).

Practice notes — 2026-09-20

What went right: good clarifying questions up front (capacity bounds, latency target); correctly reasoned through the heap alternative's lookup-cost problem when probed; grasped the stale-integer-position argument immediately once shown.

What went wrong: first design (HashMap + FIFO queue) missed that `get` must reorder; the $O(n)$ shift cost in the queue-reorder design wasn't self-identified; the key insight (map key → node) needed an explicit hint; first code attempt didn't compile, used the wrong structure, and had no eviction; second attempt was Google-assisted and still didn't compile (typos, `remove` that inserted instead of unlinking).

Score: 4/10 (10 = exceeds bar). The pattern is understood now, but independent reproduction isn't proven — re-attempt cold on a redo day before calling this one done.

Drills: pointer surgery without sentinels vs with; `unlink / insertAtHead` from memory; dry-running eviction sequences on paper.